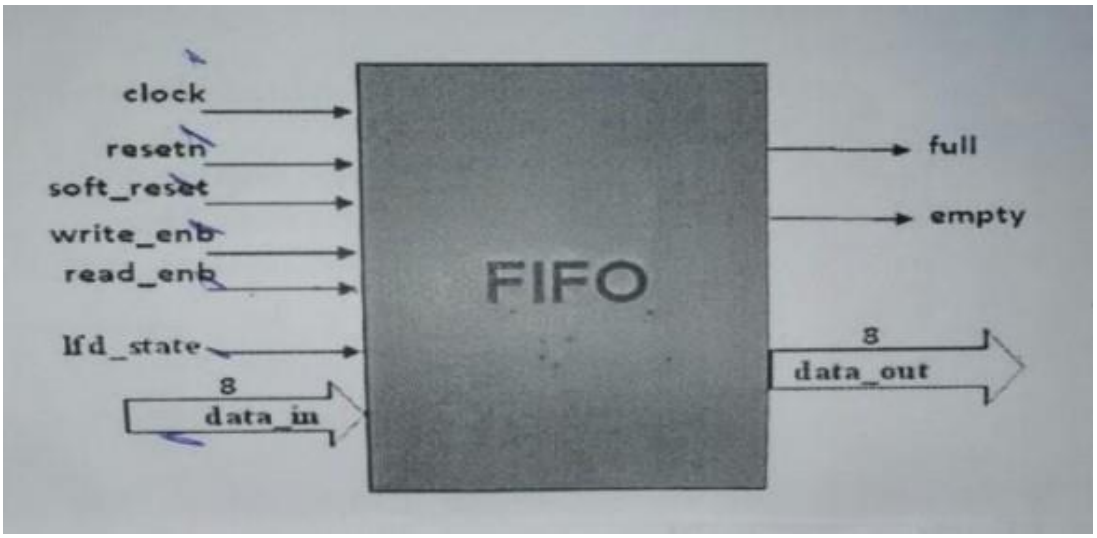
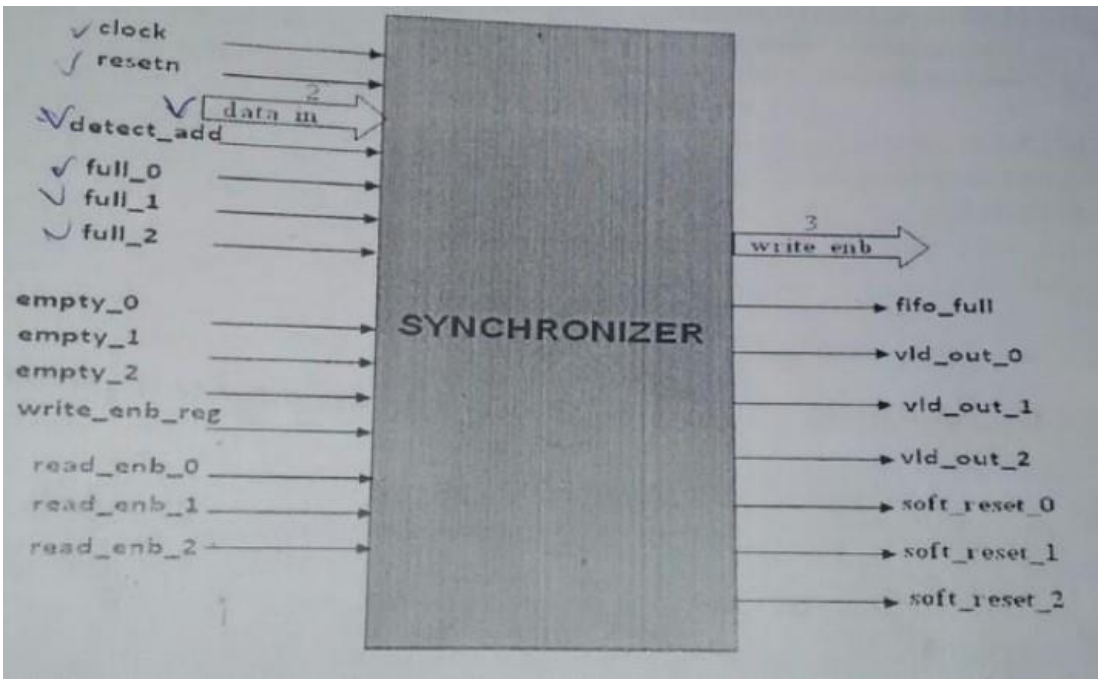
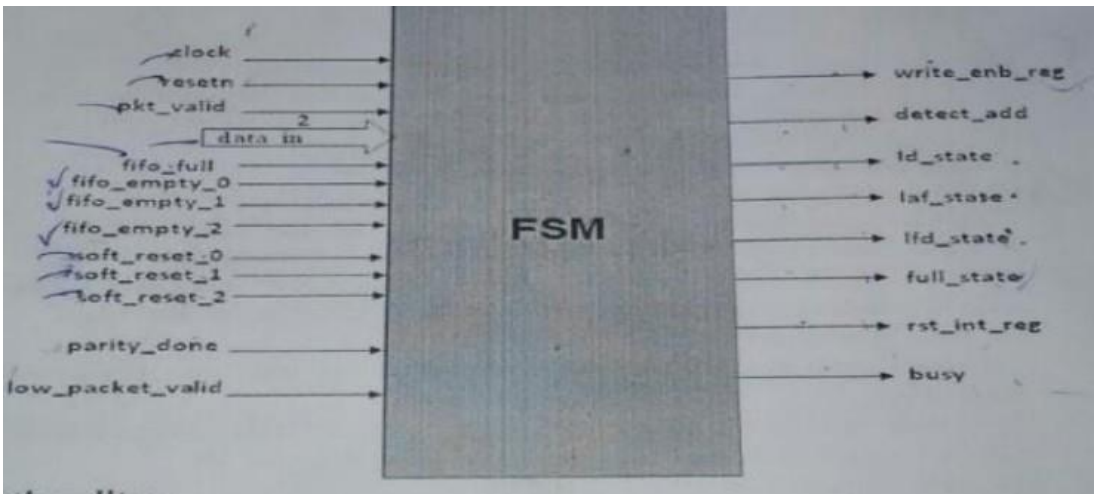
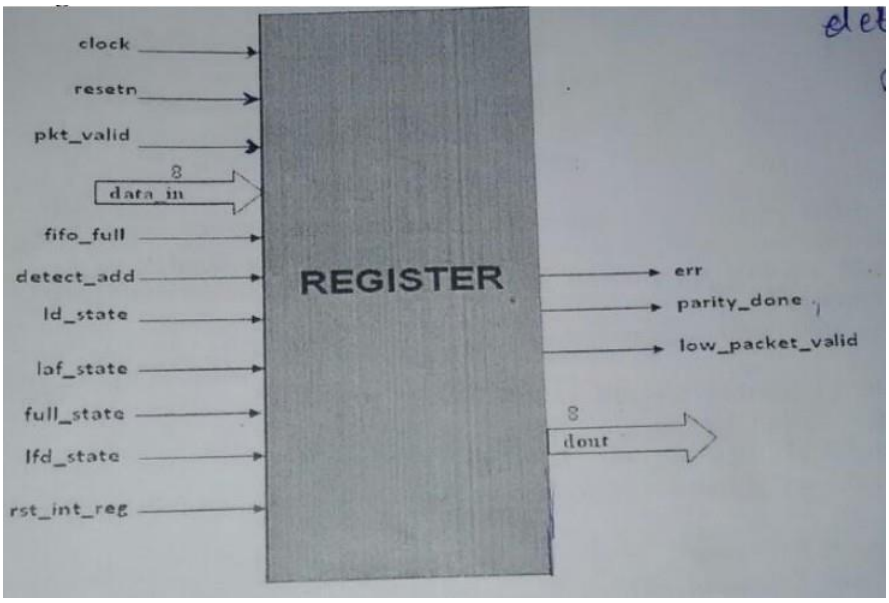

clock
resetn
soft_reset
write_enb
read_enb
lfd_state
8
data_in
FIFO
full
empty
8
data_out


clock
resetn
2
data_in
detect_add
full_0
full_1
full_2
empty_0
empty_1
empty_2
write_enb_reg
read_enb_0
read_enb_1
read_enb_2
SYNCHRONIZER
3
write_enb
fifo_full
vld_out_0
vld_out_1
vld_out_2
soft_reset_0
soft_reset_1
soft_reset_2


clock
resetn
pkt_valid
2
data_in
fifo_full
fifo_empty_0
fifo_empty_1
fifo_empty_2
soft_reset_0
soft_reset_1
soft_reset_2
parity_done
low_packet_valid
FSM
write_enb_reg
detect_add
ld_state
laf_state
lfd_state
full_state
rst_int_reg
busy



Simulation Results

7.1 Fifo

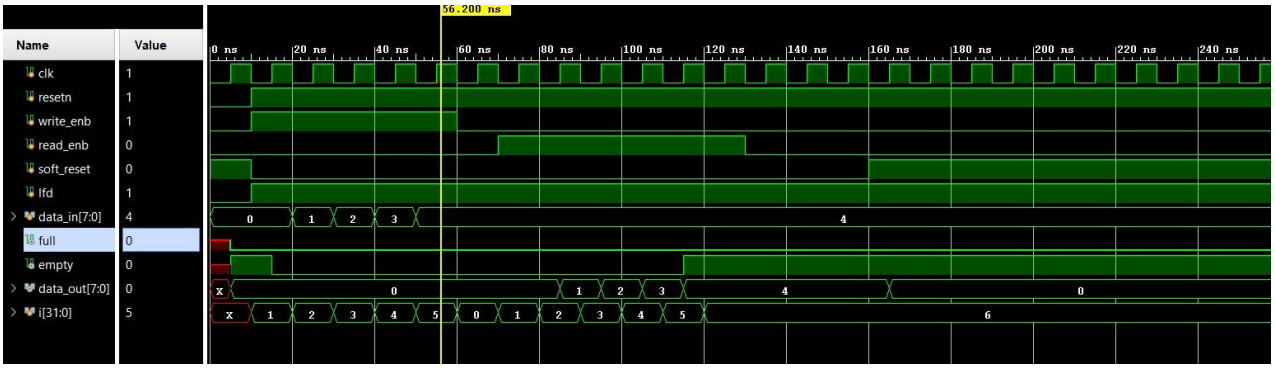


Figure 7.1: fifo waveform

7.2 Synchronizer

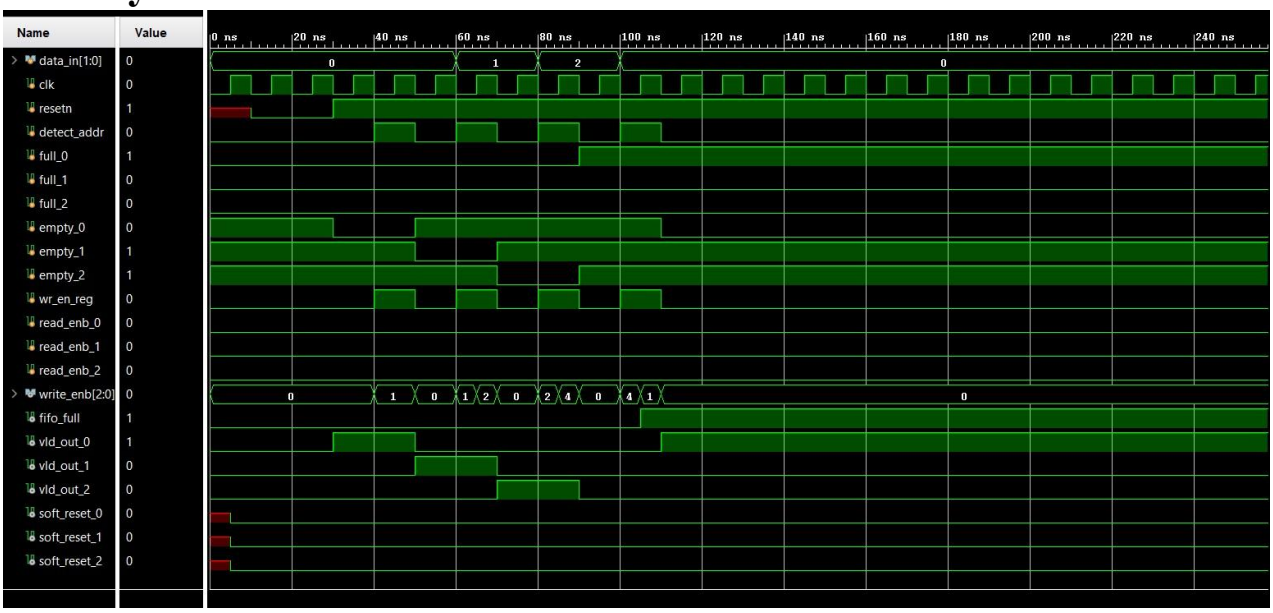


Figure 7.2: synchronizer waveform

7.3 FSM

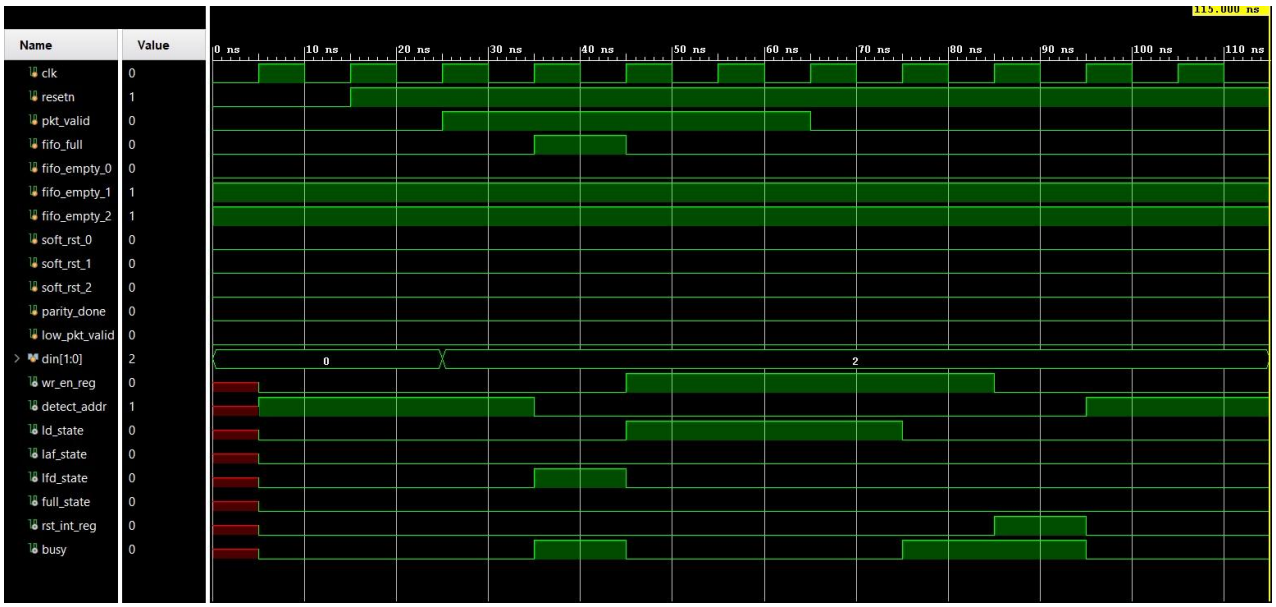


Figure 7.3: FSM waveform

7.4 Register

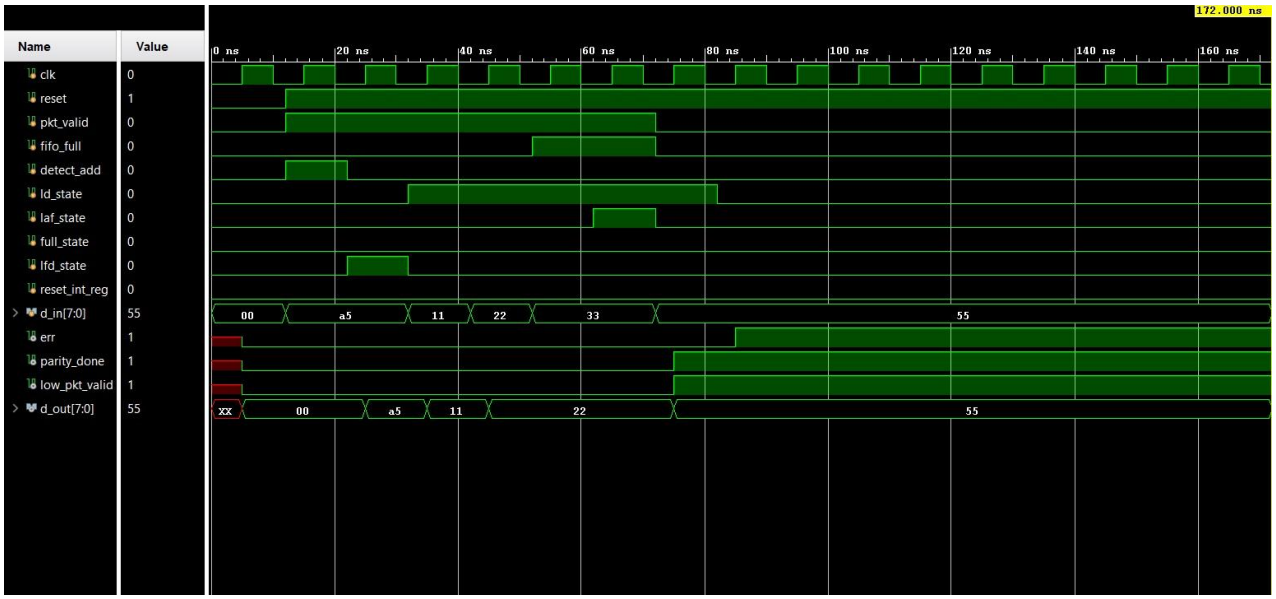


Figure 7.4: register waveform

7.5 Top module

7.5.1 Part 1

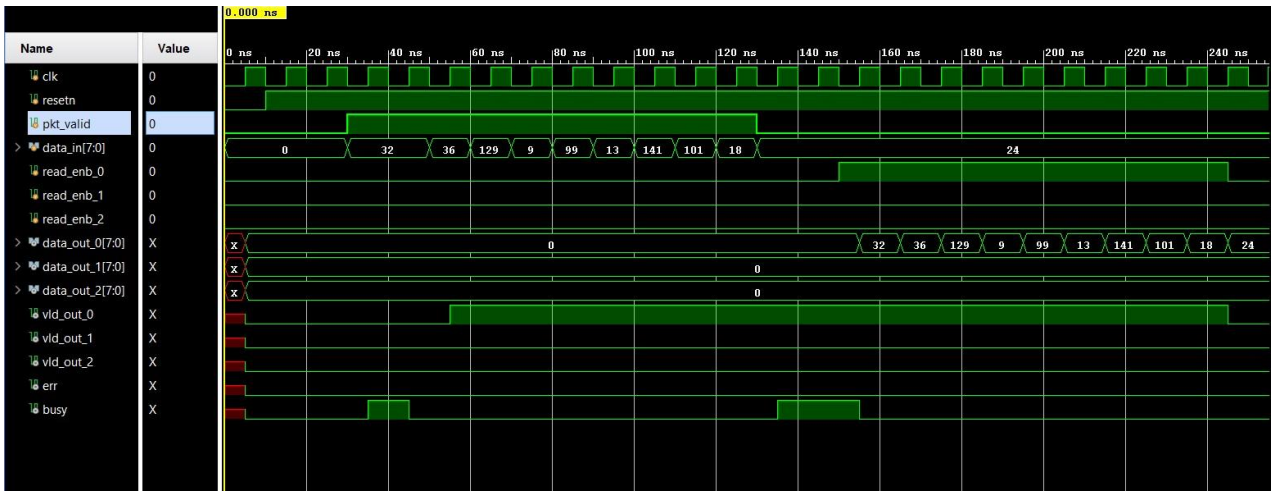


Figure 7.5: Top module waveform part 1

7.5.2 Part 2

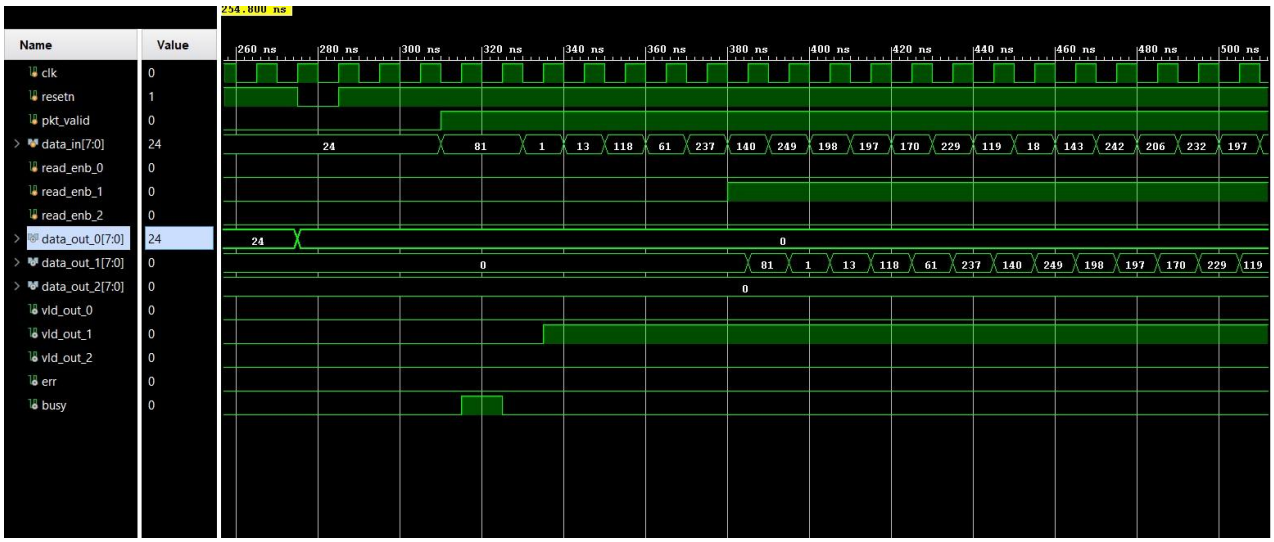


Figure 7.6: Top module waveform part 2

7.5.3 Part 3

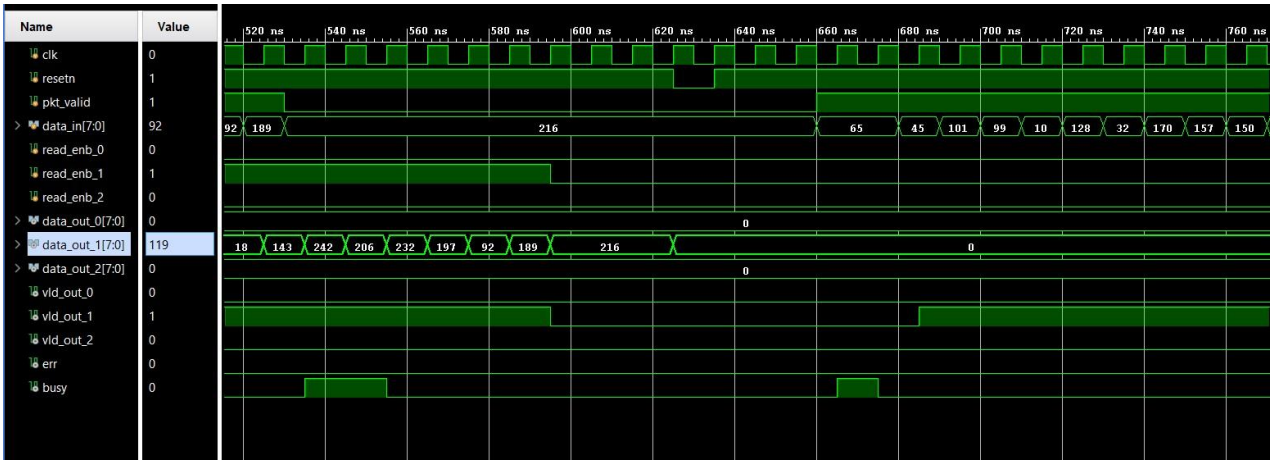


Figure 7.7: Top module waveform part 3

7.6 schematic

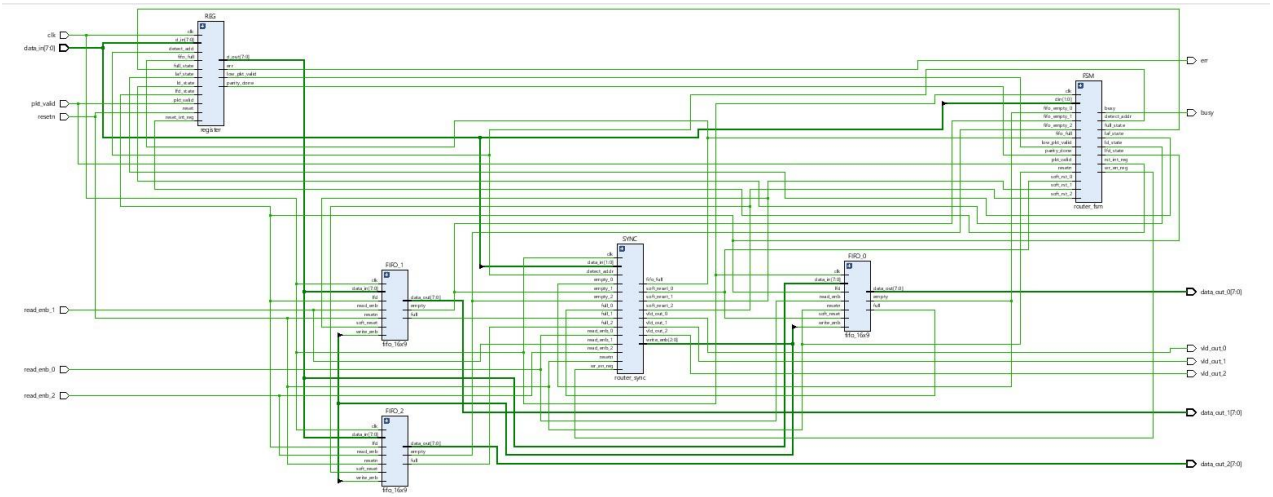


Figure 7.8: schematic

Verification

Verification is the process of validating whether the designed system meets its specified requirements. It involves applying different input stimuli to the design and observing the outputs to detect mismatches, bugs, or unexpected behavior. A well-structured verification environment helps in identifying errors early, thereby reducing development cost and time.

In this project, verification is implemented using **SystemVerilog (non-UVM methodology)**. Unlike the Universal Verification Methodology (UVM), which is a standardized and highly reusable framework, non-UVM verification uses a simpler, more direct approach. It is especially suitable for small to medium-scale designs where full UVM infrastructure is not required.

DESIGN CODE(System Verilog based):

8.1.1 FIFO:

```
Module fifo_16x9 (input logic clk,input resetn,input soft_reset,input write_enb,input
read_enb,input lfd,input [7:0] data_in,output reg [7:0] data_out,output reg full,output reg
empty);


parameter DEPTH = 16;

reg [8:0] fifo_mem [DEPTH-1:0];

reg [4:0] write_ptr, read_ptr;

reg [6:0] intCount;

reg lfd_state_t;

integer i;


always @(posedge clk)

if (!resetn)

lfd_state_t <= 1'b0;

else

lfd_state_t <= lfd;

always @(posedge clk) begin

if (!resetn)

data_out <= 8'b0;
```

```
else if (soft_reset)
data_out <= 8'b0;
else if (read_enb && !empty)
data_out <= fifo_mem[read_ptr[3:0]][7:0];
else
data_out <= data_out;
end
always @(posedge clk) begin
if (!resetn || soft_reset)
for (i = 0; i < DEPTH; i = i + 1)
fifo_mem[i] <= 9'b0;
else if (write_enb && !full) begin
fifo_mem[write_ptr[3:0]][8]   <= lfd_state_t;
fifo_mem[write_ptr[3:0]][7:0] <= data_in;
end
end
always @(posedge clk)
if (!resetn)
write_ptr <= 5'b0;
else if (write_enb && !full)
write_ptr <= write_ptr + 1'b1;
always @(posedge clk)
if (!resetn)
read_ptr <= 5'b0;
else if (read_enb && !empty)
read_ptr <= read_ptr + 1'b1;
always @(posedge clk) begin
```

```
if (!resetn)
intCount <= 0;
else if (write_enb && lfd_state_t)
intCount <= data_in[7:2] + 1'b1;
else if (read_enb && !empty && intCount != 0)
intCount <= intCount - 1'b1;
end
always @(*) begin
full  = (write_ptr == {~read_ptr[4], read_ptr[3:0]});
empty = (write_ptr == read_ptr);
end
endmodule
```

8.1.2 FSM:

```
module router_fsm (input clk,input resetn,input pkt_valid,input fifo_full,input fifo_empty_0,input
fifo_empty_1,input fifo_empty_2,input soft_rst_0,input soft_rst_1,input soft_rst_2,input
parity_done,input low_pkt_valid,input [1:0] din,output wr_en_reg,output detect_addr,output
ld_state,output laf_state,output lfd_state,output full_state,output rst_int_reg,output busy);


parameter DECODE_ADDRESS     = 3'b000;
parameter LOAD_FIRST_DATA    = 3'b001;
parameter LOAD_DATA          = 3'b010;
parameter WAIT_TILL_EMPTY    = 3'b011;
parameter CHECK_PARITY_ERROR = 3'b100;
parameter LOAD_PARITY        = 3'b101;
parameter FIFO_FULL_STATE    = 3'b110;
parameter LOAD_AFTER_FULL    = 3'b111;
reg [2:0] PS, NS;
always @(posedge clk) begin
if (!resetn)
```

```
PS <= DECODE_ADDRESS;
else if (soft_rst_0 || soft_rst_1 || soft_rst_2)
PS <= DECODE_ADDRESS;
else
PS <= NS;
end
always @(*) begin
NS = DECODE_ADDRESS;
case (PS)
DECODE_ADDRESS: begin
if ((pkt_valid && din == 2'b00 && fifo_empty_0) ||(pkt_valid && din == 2'b01 && fifo_empty_1)
||(pkt_valid && din == 2'b10 && fifo_empty_2))
NS = LOAD_FIRST_DATA;
else if ((pkt_valid && din == 2'b00 && !fifo_empty_0) ||(pkt_valid && din == 2'b01 &&
!fifo_empty_1) ||(pkt_valid && din == 2'b10 && !fifo_empty_2))
NS = WAIT_TILL_EMPTY;
else
NS = DECODE_ADDRESS;
end
LOAD_FIRST_DATA:
NS = LOAD_DATA;
LOAD_DATA: begin
if (fifo_full)
NS = FIFO_FULL_STATE;
else if (!fifo_full && !pkt_valid)
NS = LOAD_PARITY;
else
NS = LOAD_DATA;
```

```
end
WAIT_TILL_EMPTY: begin
if ((din == 2'b00 && fifo_empty_0) ||(din == 2'b01 && fifo_empty_1) ||(din == 2'b10 &&
fifo_empty_2))
NS = LOAD_FIRST_DATA;
else
NS = WAIT_TILL_EMPTY;
end
FIFO_FULL_STATE: begin
if (!fifo_full)
NS = LOAD_AFTER_FULL;
else
NS = FIFO_FULL_STATE;
end
LOAD_AFTER_FULL: begin
if (!parity_done && !low_pkt_valid)
NS = LOAD_DATA;
else if (!parity_done && low_pkt_valid)
NS = LOAD_PARITY;
else if (parity_done)
NS = DECODE_ADDRESS;
end
LOAD_PARITY:
NS = CHECK_PARITY_ERROR;
CHECK_PARITY_ERROR: begin
if (fifo_full)
NS = FIFO_FULL_STATE;
else
NS = DECODE_ADDRESS;
end
```

```
endcase
end
assign detect_addr  = (PS == DECODE_ADDRESS);
assign wr_en_reg    = (PS == LOAD_DATA || PS == LOAD_PARITY || PS == LOAD_AFTER_FULL);
assign ld_state     = (PS == LOAD_DATA);
assign laf_state    = (PS == LOAD_AFTER_FULL);
assign lfd_state    = (PS == LOAD_FIRST_DATA);
assign full_state   = (PS == FIFO_FULL_STATE);
assign rst_int_reg  = (PS == CHECK_PARITY_ERROR);
assign busy         = (PS == LOAD_FIRST_DATA || PS == LOAD_PARITY ||PS == FIFO_FULL_STATE ||
PS == LOAD_AFTER_FULL ||PS == WAIT_TILL_EMPTY || PS == CHECK_PARITY_ERROR);
endmodule
```

8.1.3 REGISTER:

```
module register (input clk,input reset,input pkt_valid,input [7:0] d_in,input fifo_full,input
detect_add,input ld_state,input laf_state,input full_state,input lfd_state,input
reset_int_reg,output reg [7:0] d_out,output reg err,output reg parity_done,output reg
low_pkt_valid);


reg [7:0] header_byte, internal_reg, internal_parity, packet_parity;
always @(posedge clk) begin
if (!reset)
parity_done <= 0;
else if (detect_add)
parity_done <= 0;
else if ((ld_state && (~fifo_full) && (~pkt_valid)) || (laf_state && low_pkt_valid &&
(~parity_done)))
parity_done <= 1;
```

```
end
always @(posedge clk) begin
if (!reset)
low_pkt_valid <= 0;
else if (reset_int_reg)
low_pkt_valid <= 0;
else if (ld_state && ~pkt_valid)
low_pkt_valid <= 1;
end
always @(posedge clk) begin
if (!reset) begin
d_out        <= 8'd0;
header_byte   <= 8'd0;
internal_reg  <= 8'd0;
end
else if (detect_add && pkt_valid && d_in[1:0] != 2'b11)
header_byte <= d_in;
else if (lfd_state)
d_out <= header_byte;
else if (ld_state && ~fifo_full)
d_out <= d_in;
else if (ld_state && fifo_full)
internal_reg <= d_in;
else if (laf_state)
d_out <= internal_reg;
end
always @(posedge clk) begin
if (!reset)
internal_parity <= 8'd0;
```

```
else if (detect_add)
internal_parity <= 8'd0;
else if (lfd_state && pkt_valid)
internal_parity <= internal_parity ^ header_byte;
else if (ld_state && pkt_valid && ~full_state)
internal_parity <= internal_parity ^ d_in;
else
internal_parity <= internal_parity;
end
always @(posedge clk) begin
if (!reset)
err <= 0;
else if (parity_done) begin
if (internal_parity == packet_parity)
err <= 0;
else
err <= 1;
end
else
err <= 0;
end
always @(posedge clk) begin
if (!reset)
packet_parity <= 8'd0;
else if (detect_add)
packet_parity <= 8'd0;
else if ((ld_state && !fifo_full && !pkt_valid) || (laf_state && ~parity_done && low_pkt_valid))
```

```
packet_parity <= d_in;

end

endmodule
```

8.1.4 SYNCHRONISER:

```
module router_sync (input  clk,input  resetn,input  [1:0] data_in,input  detect_addr,input
full_0,input  full_1,input  full_2,input  empty_0,input  empty_1,input  empty_2,input
wr_en_reg,input  read_enb_0,

input  read_enb_1,input  read_enb_2,output reg [2:0] write_enb,output reg fifo_full,output
vld_out_0,output vld_out_1,output vld_out_2,output reg soft_reset_0,output reg
soft_reset_1,output reg soft_reset_2);

reg [1:0] tmp_din;

reg [5:0] count0, count1, count2;

always @(posedge clk or negedge resetn) begin

if (!resetn)

tmp_din <= 2'b00;

else if (detect_addr)

tmp_din <= data_in;

end

always @(*) begin

case (tmp_din)

2'b00: begin

fifo_full = full_0;

write_enb = (wr_en_reg) ? 3'b001 : 3'b000;

end

2'b01: begin

fifo_full = full_1;

write_enb = (wr_en_reg) ? 3'b010 : 3'b000;

end

2'b10: begin
```

```
fifo_full = full_2;
write_enb = (wr_en_reg) ? 3'b100 : 3'b000;
end

default: begin
fifo_full = 1'b0;
write_enb = 3'b000;
end
endcase
end
assign vld_out_0 = ~empty_0;
assign vld_out_1 = ~empty_1;
assign vld_out_2 = ~empty_2;
always @(posedge clk or negedge resetn) begin
if (!resetn) begin
count0 <= 0;
soft_reset_0 <= 0;
end
else if (vld_out_0) begin
if (!read_enb_0) begin
if (count0 == 29) begin
soft_reset_0 <= 1;
count0 <= 0;
end
else begin
soft_reset_0 <= 0;
count0 <= count0 + 1;
```

```
end
end
else begin
count0 <= 0;
soft_reset_0 <= 0;
end
end
else begin
count0 <= 0;
soft_reset_0 <= 0;
end
end
always @(posedge clk or negedge resetn) begin
if (!resetn) begin
count1 <= 0;
soft_reset_1 <= 0;
end
else if (vld_out_1) begin
if (!read_enb_1) begin
if (count1 == 29) begin
soft_reset_1 <= 1;
count1 <= 0;
end
else begin
soft_reset_1 <= 0;
count1 <= count1 + 1;
end
end
else begin
```

```
count1 <= 0;
soft_reset_1 <= 0;
end
end
else begin
count1 <= 0;
soft_reset_1 <= 0;
end
end
always @(posedge clk or negedge resetn) begin
if (!resetn) begin
count2 <= 0;
soft_reset_2 <= 0;
end
else if (vld_out_2) begin
if (!read_enb_2) begin
if (count2 == 29) begin
soft_reset_2 <= 1;
count2 <= 0;
end
else begin
soft_reset_2 <= 0;
count2 <= count2 + 1;
end
end
else begin
count2 <= 0;
```

```
soft_reset_2 <= 0;
end
end
else begin
count2 <= 0;
soft_reset_2 <= 0;
end
end
endmodule
```

8.1.5 ROUTER:

```
 `include "fifo.sv"
`include "synchroniser.sv"
`include "register.sv"
`include "fsm.sv"
module router_1x3 (
  input  logic clk,
  input  logic resetn,

  input  logic pkt_valid,
  input  logic [7:0] data_in,

  input  logic read_enb_0,
  input  logic read_enb_1,
  input  logic read_enb_2,

  output logic vld_out_0,
  output logic vld_out_1,
  output logic vld_out_2,
```

```
  output logic err,
  output logic busy,

  output logic [7:0] data_out_0,
  output logic [7:0] data_out_1,
  output logic [7:0] data_out_2
);


   logic soft_reset_0, full_0, empty_0;
   logic soft_reset_1, full_1, empty_1;
    logic soft_reset_2, full_2, empty_2;
   logic fifo_full, detect_addr, ld_state, laf_state;
   logic full_state, lfd_state, reset_int_reg;
   logic parity_done, low_pkt_valid, wr_en_reg;
   logic [2:0] write_enb;
   logic [7:0] reg_dout;


    fifo_16x9 FIFO_0 (
        .clk(clk),
        .resetn(resetn),
        .soft_reset(soft_reset_0),
        .write_enb(write_enb[0]),
        .read_enb(read_enb_0),
        .lfd(lfd_state),
```

```
    .data_in(reg_dout),
    .data_out(data_out_0),
    .full(full_0),
    .empty(empty_0)
);

fifo_16x9 FIFO_1 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_1),
    .write_enb(write_enb[1]),
    .read_enb(read_enb_1),
    .lfd(lfd_state),
    .data_in(reg_dout),
    .data_out(data_out_1),
    .full(full_1),
    .empty(empty_1)
);

fifo_16x9 FIFO_2 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_2),
    .write_enb(write_enb[2]),
    .read_enb(read_enb_2),
    .lfd(lfd_state),
    .data_in(reg_dout),
    .data_out(data_out_2),
    .full(full_2),
```

```
    .empty(empty_2)
);


router_sync SYNC (
    .clk(clk),
    .resetn(resetn),
    .detect_addr(detect_addr),
    .wr_en_reg(wr_en_reg),
    .data_in(data_in[1:0]),
    .full_0(full_0),
    .full_1(full_1),
    .full_2(full_2),
    .empty_0(empty_0),
    .empty_1(empty_1),
    .empty_2(empty_2),
    .read_enb_0(read_enb_0),
    .read_enb_1(read_enb_1),
    .read_enb_2(read_enb_2),
    .write_enb(write_enb),
    .fifo_full(fifo_full),
    .vld_out_0(vld_out_0),
    .vld_out_1(vld_out_1),
    .vld_out_2(vld_out_2),
    .soft_reset_0(soft_reset_0),
    .soft_reset_1(soft_reset_1),
    .soft_reset_2(soft_reset_2)
```

```
);


register REG (
    .clk(clk),
    .reset(resetn),
    .pkt_valid(pkt_valid),
    .d_in(data_in),
    .fifo_full(fifo_full),
    .detect_add(detect_addr),
    .ld_state(ld_state),
    .laf_state(laf_state),
    .full_state(full_state),
    .lfd_state(lfd_state),
    .reset_int_reg(reset_int_reg),
    .d_out(reg_dout),
    .err(err),
    .parity_done(parity_done),
    .low_pkt_valid(low_pkt_valid)
);


router_fsm FSM (
    .clk(clk),
    .resetn(resetn),
    .pkt_valid(pkt_valid),
    .fifo_full(fifo_full),
    .fifo_empty_0(empty_0),
    .fifo_empty_1(empty_1),
```

```
        .fifo_empty_2(empty_2),
        .soft_rst_0(soft_reset_0),
        .soft_rst_1(soft_reset_1),
        .soft_rst_2(soft_reset_2),
        .parity_done(parity_done),
        .low_pkt_valid(low_pkt_valid),
        .din(data_in[1:0]),
        .wr_en_reg(wr_en_reg),
        .detect_addr(detect_addr),
        .ld_state(ld_state),
        .laf_state(laf_state),
        .lfd_state(lfd_state),
        .full_state(full_state),
        .rst_int_reg(reset_int_reg),
        .busy(busy)
    );

endmodule
```

The verification environment is built using fundamental SystemVerilog constructs such as:

8.2.1 TRANSACTION

```
class router_transaction;

  // Packet structure
  rand bit [7:0] header;
```

```
rand bit [7:0] payload[];
rand bit [7:0] parity;

rand bit [1:0] dest;

// Monitor output capture
bit [7:0] data_out_0;
bit [7:0] data_out_1;
bit [7:0] data_out_2;

int ch;

// Payload constraint
constraint payload_size { payload.size inside {[1:8]}; }

constraint dest_c { dest inside {2'b00,2'b01,2'b10}; }

// Packet formation
function void post_randomize();

  header[1:0] = dest;
  header[7:2] = payload.size;

  parity = header;
  foreach(payload[i])
    parity ^= payload[i];

endfunction
```

```
  function void display();
    $display("---- PACKET ----");
    $display("DEST = %0d", dest);
    $display("HEADER = %0h", header);

    foreach(payload[i])
      $display("PAYLOAD[%0d] = %0h", i, payload[i]);

    $display("PARITY = %0h", parity);
  endfunction

endclass
```

8.2.2 GENERATOR

```
class router_generator;

  mailbox gen2drv;

  int repeat_count;

  function new(mailbox gen2drv);
    this.gen2drv = gen2drv;
  endfunction


  task main();
```

```
    router_transaction trans;

    repeat(repeat_count) begin

      trans = new();
      assert(trans.randomize());

      $display("[GEN]");
      trans.display();

      gen2drv.put(trans);

    end

  endtask

endclass
```

8.2.3 INTERFACE

```
interface router_intf(
    input bit clk,
    input bit resetn
);

logic pkt_valid;
logic [7:0] data_in;

logic read_enb_0;
logic read_enb_1;
logic read_enb_2;
```

```
logic vld_out_0;
logic vld_out_1;
logic vld_out_2;

logic [7:0] data_out_0;
logic [7:0] data_out_1;
logic [7:0] data_out_2;

logic err;
logic busy;

endinterface
```

8.2.4 MONITOR

```
class router_monitor;

  virtual router_intf vif;
  mailbox mon2scb;

  function new(virtual router_intf vif, mailbox mon2scb);
    this.vif = vif;
    this.mon2scb = mon2scb;
  endfunction

  task main();

    forever begin

      @(posedge vif.clk);
```

```
    if(vif.vld_out_0 && vif.read_enb_0) begin
      router_transaction t = new();
      t.dest = 0;
      t.header = vif.data_out_0;
      mon2scb.put(t);
      $display("[MON] FIFO0 %h",vif.data_out_0);
    end

    if(vif.vld_out_1 && vif.read_enb_1) begin
      router_transaction t = new();
      t.dest = 1;
      t.header = vif.data_out_1;
      mon2scb.put(t);
      $display("[MON] FIFO1 %h",vif.data_out_1);
    end

    if(vif.vld_out_2 && vif.read_enb_2) begin
      router_transaction t = new();
      t.dest = 2;
      t.header = vif.data_out_2;
      mon2scb.put(t);
      $display("[MON] FIFO2 %h",vif.data_out_2);
    end

  end

 endtask

endclass
```

8.2.5 DRIVER

```
class router_driver;

  virtual router_intf vif;
  mailbox gen2drv;

  function new(virtual router_intf vif, mailbox gen2drv);
    this.vif = vif;
    this.gen2drv = gen2drv;
  endfunction

  task reset_phase();

    wait(!vif.resetn);

    vif.pkt_valid <= 0;
    vif.data_in   <= 0;

    vif.read_enb_0 <= 0;
    vif.read_enb_1 <= 0;
    vif.read_enb_2 <= 0;

    wait(vif.resetn);

  endtask


  task main();
```

```
router_transaction trans;

forever begin

  gen2drv.get(trans);

  // HEADER
  @(posedge vif.clk);
  vif.pkt_valid <= 1;
  vif.data_in   <= trans.header;

  // PAYLOAD
  foreach(trans.payload[i]) begin
    @(posedge vif.clk);
    vif.data_in <= trans.payload[i];
  end

  // PARITY
  @(posedge vif.clk);
  vif.pkt_valid <= 0;
  vif.data_in   <= trans.parity;

  @(posedge vif.clk);

  // READ FIFO
  case(trans.dest)

    2'b00: begin
      vif.read_enb_0 <= 1;
      while(vif.vld_out_0)
```

```
        @(posedge vif.clk);
      vif.read_enb_0 <= 0;
    end

    2'b01: begin
      vif.read_enb_1 <= 1;
      while(vif.vld_out_1)
        @(posedge vif.clk);
      vif.read_enb_1 <= 0;
    end

    2'b10: begin
      vif.read_enb_2 <= 1;
      while(vif.vld_out_2)
        @(posedge vif.clk);
      vif.read_enb_2 <= 0;
    end

   endcase

  end

 endtask

endclass
```

8.2.6 SCOREBOARD

```
class router_scoreboard;
```

```
mailbox mon2scb;
int no_transactions;

function new(mailbox mon2scb);
  this.mon2scb = mon2scb;
endfunction

task check(router_transaction trans);

  case(trans.ch)

    0: begin
      if(trans.header[1:0] == 2'b00)
        $display("[SCB] PASS FIFO0 %0h", trans.data_out_0);
      else
        $error("[SCB] FAIL FIFO0 routing");
    end

    1: begin
      if(trans.header[1:0] == 2'b01)
        $display("[SCB] PASS FIFO1 %0h", trans.data_out_1);
      else
        $error("[SCB] FAIL FIFO1 routing");
    end

    2: begin
      if(trans.header[1:0] == 2'b10)
        $display("[SCB] PASS FIFO2 %0h", trans.data_out_2);
      else
        $error("[SCB] FAIL FIFO2 routing");
    end
```

```
    endcase

    no_transactions++;

  endtask


  task main();

    router_transaction trans;

    forever begin
      mon2scb.get(trans);
      check(trans);
    end

  endtask

endclass
```

8.2.7 ENVIRONMENT

```
`include "transaction.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
class router_environment;
```

```
router_generator gen;
router_driver drv;
router_monitor mon;
router_scoreboard scb;

mailbox gen2drv;
mailbox mon2scb;

virtual router_intf vif;

function new(virtual router_intf vif);

  this.vif = vif;

  gen2drv = new();
  mon2scb = new();

  gen = new(gen2drv);
  drv = new(vif,gen2drv);
  mon = new(vif,mon2scb);
  scb = new(mon2scb);

endfunction


task run();

  drv.reset_phase();

  fork
    gen.main();
```

```
    drv.main();
    mon.main();
    scb.main();
  join_none

 endtask

endclass
```

8.2.8 TEST

```
program test(router_intf rif);

 router_environment env;

 initial begin

  env = new(rif);

  env.gen.repeat_count = 20;

  env.run();

  #20000 $finish;

 end

endprogram
```

8.2.9 TOP

```
`include "interface.sv"
`include "environment.sv"
`include "test.sv"
module router_tb_top;

  bit clk = 0;
  bit resetn = 0;

  always #5 clk = ~clk;

  initial begin
    resetn = 0;
    #20 resetn = 1;
  end

  router_intf rif(clk,resetn);

  router_1x3 DUT(
    .clk(clk),
    .resetn(resetn),
    .pkt_valid(rif.pkt_valid),
    .data_in(rif.data_in),
    .read_enb_0(rif.read_enb_0),
    .read_enb_1(rif.read_enb_1),
    .read_enb_2(rif.read_enb_2),
    .vld_out_0(rif.vld_out_0),
    .vld_out_1(rif.vld_out_1),
    .vld_out_2(rif.vld_out_2),
    .err(rif.err),
    .busy(rif.busy),
    .data_out_0(rif.data_out_0),
```

```
    .data_out_1(rif.data_out_1),
    .data_out_2(rif.data_out_2)
  );

  test t1(rif);

endmodule
```

OUTPUT:

```
[GEN]
---- PACKET ----
DEST = 2
HEADER = a
PAYLOAD[0] = a9
PAYLOAD[1] = 5b
PARITY = f8
[GEN]
---- PACKET ----
DEST = 2
HEADER = 6
PAYLOAD[0] = b8
PARITY = be
[GEN]
---- PACKET ----
DEST = 2
HEADER = 1a
PAYLOAD[0] = ee
PAYLOAD[1] = 7f
PAYLOAD[2] = f5
PAYLOAD[3] = 30
PAYLOAD[4] = 57
PAYLOAD[5] = 32
PARITY = 2b
[GEN]
```

```
---- PACKET ----
DEST = 1
HEADER = 9
PAYLOAD[0] = aa
PAYLOAD[1] = c
PARITY = af
[GEN]
---- PACKET ----
DEST = 1
HEADER = 5
PAYLOAD[0] = ab
PARITY = ae
[GEN]
---- PACKET ----
DEST = 1
HEADER = 1d
PAYLOAD[0] = b4
PAYLOAD[1] = ec
PAYLOAD[2] = c8
PAYLOAD[3] = 41
PAYLOAD[4] = 45
PAYLOAD[5] = 5a
PAYLOAD[6] = e1
PARITY = 32
[GEN]
---- PACKET ----
DEST = 2
HEADER = 6
PAYLOAD[0] = f9
PARITY = ff
[GEN]
---- PACKET ----
DEST = 0
HEADER = 14
PAYLOAD[0] = 1f
PAYLOAD[1] = 69
PAYLOAD[2] = a8
PAYLOAD[3] = d2
PAYLOAD[4] = 30
PARITY = 28
[GEN]
---- PACKET ----
DEST = 0
HEADER = 10
```

```
PAYLOAD[0] = f6
PAYLOAD[1] = 7
PAYLOAD[2] = 9c
PAYLOAD[3] = 95
PARITY = e8
[GEN]
---- PACKET ----
DEST = 2
HEADER = 12
PAYLOAD[0] = 1f
PAYLOAD[1] = 86
PAYLOAD[2] = 11
PAYLOAD[3] = d7
PARITY = 4d
[GEN]
---- PACKET ----
DEST = 1
HEADER = 21
PAYLOAD[0] = f5
PAYLOAD[1] = e7
PAYLOAD[2] = d6
PAYLOAD[3] = f4
PAYLOAD[4] = 8b
PAYLOAD[5] = 7f
PAYLOAD[6] = 7a
PAYLOAD[7] = 6d
PARITY = f2
[GEN]
---- PACKET ----
DEST = 0
HEADER = 14
PAYLOAD[0] = 55
PAYLOAD[1] = ea
PAYLOAD[2] = 95
PAYLOAD[3] = 7e
PAYLOAD[4] = e8
PARITY = a8
[GEN]
---- PACKET ----
DEST = 2
```

HEADER = 16
PAYLOAD[0] = 2b
PAYLOAD[1] = 79
PAYLOAD[2] = b0
PAYLOAD[3] = 8f
PAYLOAD[4] = 1c
PARITY = 67
[GEN]
---- PACKET ----
DEST = 1
HEADER = 21
PAYLOAD[0] = 13
PAYLOAD[1] = 9c
PAYLOAD[2] = ed
PAYLOAD[3] = 2f
PAYLOAD[4] = a3
PAYLOAD[5] = b3
PAYLOAD[6] = 98
PAYLOAD[7] = c5
PARITY = 21
[GEN]
---- PACKET ----
DEST = 1
HEADER = 21
PAYLOAD[0] = 7d
PAYLOAD[1] = ce
PAYLOAD[2] = 43
PAYLOAD[3] = a2
PAYLOAD[4] = c1
PAYLOAD[5] = e0
PAYLOAD[6] = ec
PAYLOAD[7] = b8
PARITY = 6
[GEN]
---- PACKET ----
DEST = 2
HEADER = 6
PAYLOAD[0] = 5f
PARITY = 59
[GEN]
---- PACKET ----
DEST = 0
HEADER = 20
PAYLOAD[0] = 96

```
PAYLOAD[1] = c3
PAYLOAD[2] = 27
PAYLOAD[3] = 3e
PAYLOAD[4] = 27
PAYLOAD[5] = 8
PAYLOAD[6] = 1a
PAYLOAD[7] = 63
PARITY = 3a
[GEN]
---- PACKET ----
DEST = 0
HEADER = 14
PAYLOAD[0] = 99
PAYLOAD[1] = f2
PAYLOAD[2] = 22
PAYLOAD[3] = 9f
PAYLOAD[4] = 7e
PARITY = bc
[GEN]
---- PACKET ----
DEST = 1
HEADER = 11
PAYLOAD[0] = 19
PAYLOAD[1] = b5
PAYLOAD[2] = f1
PAYLOAD[3] = 3a
PARITY = 76
[GEN]
---- PACKET ----
DEST = 1
HEADER = 1d
PAYLOAD[0] = cb
PAYLOAD[1] = b5
PAYLOAD[2] = d8
PAYLOAD[3] = ec
PAYLOAD[4] = 7c
PAYLOAD[5] = 60
PAYLOAD[6] = 1
PARITY = 4a
[MON] FIFO2 0a
```

```
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 75 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 5b
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 85 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 06
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 145 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 1a
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 255 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 7f
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 265 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 f5
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 275 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 30
[SCB] PASS FIFO0 0
[MON] FIFO2 57
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 295 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 32
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 305 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 09
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 375 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 0c
[SCB] PASS FIFO0 0
[MON] FIFO1 05
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 445 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 1d
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 565 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 ec
[SCB] PASS FIFO0 0
[MON] FIFO1 c8
[SCB] PASS FIFO0 0
[MON] FIFO1 41
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 595 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 45
```

```
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 605 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 5a
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 615 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 e1
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 625 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 06
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 685 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 14
[SCB] PASS FIFO0 0
[MON] FIFO0 69
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 795 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 a8
[SCB] PASS FIFO0 0
[MON] FIFO0 d2
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 815 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 30
[SCB] PASS FIFO0 0
[MON] FIFO0 10
[SCB] PASS FIFO0 0
[MON] FIFO0 07
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 925 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 9c
[SCB] PASS FIFO0 0
[MON] FIFO0 95
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 945 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 12
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1035 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 86
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1045 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 11
```

```
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1055 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 d7
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1065 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 21
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1195 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 e7
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1205 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 d6
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1215 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 f4
[SCB] PASS FIFO0 0
[MON] FIFO1 8b
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1235 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 7f
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1245 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 7a
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1255 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 6d
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1265 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 14
[SCB] PASS FIFO0 0
[MON] FIFO0 ea
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1375 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 95
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1385 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 7e
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1395 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 e8
[SCB] PASS FIFO0 0
[MON] FIFO2 16
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1505 ns
[SCB] FAIL FIFO0 routing
```

```
[MON] FIFO2 79
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1515 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 b0
[SCB] PASS FIFO0 0
[MON] FIFO2 8f
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1535 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO2 1c
[SCB] PASS FIFO0 0
[MON] FIFO1 21
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1675 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 9c
[SCB] PASS FIFO0 0
[MON] FIFO1 ed
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1695 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 2f
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1705 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 a3
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1715 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 b3
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1725 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 98
[SCB] PASS FIFO0 0
[MON] FIFO1 c5
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1745 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 21
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1875 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 ce
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1885 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 43
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1895 ns
```

```
[SCB] FAIL FIFO0 routing
[MON] FIFO1 a2
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1905 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 c1
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 1915 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 e0
[SCB] PASS FIFO0 0
[MON] FIFO1 ec
[SCB] PASS FIFO0 0
[MON] FIFO1 b8
[SCB] PASS FIFO0 0
[MON] FIFO2 06
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2005 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 20
[SCB] PASS FIFO0 0
[MON] FIFO0 c3
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2145 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 27
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2155 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 3e
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2165 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 27
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2175 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 08
[SCB] PASS FIFO0 0
[MON] FIFO0 1a
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2195 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 63
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2205 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 14
[SCB] PASS FIFO0 0
[MON] FIFO0 f2
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2315 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 22
```

```
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2325 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 9f
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2335 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO0 7e
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2345 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 11
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2435 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 b5
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2445 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 f1
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2455 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 3a
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2465 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 1d
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2585 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 b5
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2595 ns
[SCB] FAIL FIFO0 routing
[MON] FIFO1 d8
[SCB] PASS FIFO0 0
[MON] FIFO1 ec
[SCB] PASS FIFO0 0
[MON] FIFO1 7c
[SCB] PASS FIFO0 0
[MON] FIFO1 60
[SCB] PASS FIFO0 0
[MON] FIFO1 01
Error: "scoreboard.sv", 18: $unit::\router_scoreboard::check : at time 2645 ns
[SCB] FAIL FIFO0 routing
$finish called from file "test.sv", line 13.
$finish at simulation time
```

The simulation output confirms that:

- The verification environment (generator, driver, monitor, and scoreboard) is functioning correctly.
- The router is able to route packets to different FIFOs based on destination address.
- The presence of FAIL cases highlights the effectiveness of the verification setup in identifying design or synchronization issues.

Thus, the obtained output validates the working of the **1×3 router verification framework**, ensuring both functional checking and error detection.

DESIGN

A.1 FIFO

```
module fifo_16x9 (input clk,input resetn,input soft_reset,input write_enb,input
read_enb,input lfd,input [7:0] data_in,output reg [7:0] data_out,output reg full,output reg
empty);

parameter DEPTH = 16; reg
[8:0] fifo_mem [DEPTH-1:0];
reg [4:0] write_ptr, read_ptr; reg
[6:0] intCount; reg lfd_state_t;
integer i;

always @(posedge clk)
if (!resetn) lfd_state_t
<= 1'b0; else
lfd_state_t <= lfd; always @(posedge clk)
begin if (!resetn) data_out <= 8'b0; else if
(soft_reset) data_out <= 8'b0; else if
(read_enb && !empty) data_out <=
fifo_mem[read_ptr[3:0]][7:0]; else
data_out <= data_out;
end always @(posedge clk) begin if (!resetn
|| soft_reset) for (i = 0; i < DEPTH; i = i + 1)
fifo_mem[i] <= 9'b0; else if (write_enb &&
!full) begin fifo_mem[write_ptr[3:0]][8] <=
```

```
lfd_state_t;     fifo_mem[write_ptr[3:0]][7:0]
<= data_in; end
end
always @(posedge clk) if
(!resetn) write_ptr <= 5'b0;
else if (write_enb && !full)
write_ptr <= write_ptr + 1'b1;
always @(posedge clk) if
(!resetn) read_ptr <= 5'b0;
else if (read_enb && !empty)
read_ptr <= read_ptr + 1'b1;
always @(posedge clk) begin
if (!resetn) intCount <= 0;
else if (write_enb && lfd_state_t)
intCount <= data_in[7:2] + 1'b1;  // Initialize on header write
else if (read_enb && !empty && intCount != 0) intCount <=
intCount - 1'b1;
end
always @(*) begin
full  = (write_ptr == {~read_ptr[4], read_ptr[3:0]}); empty
= (write_ptr == read_ptr);
end
endmodule
```

TEST BENCH:

```
module fifo_16x9_tb;

reg clk = 0, resetn, write_enb, read_enb, soft_reset, lfd;
reg [7:0] data_in; wire full, empty; wire [7:0]
data_out;

fifo_16x9 dut
(.clk(clk),.resetn(resetn),.write_enb(write_enb),.read_enb(read_enb),.soft_reset(soft_reset),.lf
d(lfd),.data_in(data_in),.full(full),.empty(empty),.data_out(data_out));

always #5 clk = ~clk;
integer i;
initial begin
resetn     = 0;
write_enb  = 0;
```

```
read_enb   = 0;
soft_reset = 1;
data_in    = 0; lfd
= 0;
#10 resetn = 1;
soft_reset = 0;

for (i = 0; i < 5; i = i + 1) begin
@(negedge clk);
write_enb   =   1;
read_enb    =   0;
data_in   = i;
lfd      = 1;
end
@(negedge clk) write_enb = 0; for
(i = 0; i < 6; i = i + 1) begin
@(negedge clk);
write_enb   =   0;
read_enb    =   1;
end
@(negedge clk) read_enb = 0;
#30 soft_reset = 0;
@(negedge clk);
soft_reset = 1;
#100 $finish; end
endmodule
```

A.2 FSM

```
module router_fsm (input clk,input resetn,input pkt_valid,input fifo_full,input
fifo_empty_0,input fifo_empty_1,input fifo_empty_2,input soft_rst_0,input soft_rst_1,input
soft_rst_2,input parity_done,input low_pkt_valid,input [1:0] din,output wr_en_reg,output
detect_addr,output ld_state,output laf_state,output lfd_state,output full_state,output
rst_int_reg,output busy);

parameter DECODE_ADDRESS     = 3'b000;
parameter LOAD_FIRST_DATA    = 3'b001;
parameter LOAD_DATA          = 3'b010; parameter
WAIT_TILL_EMPTY    = 3'b011; parameter
CHECK_PARITY_ERROR = 3'b100; parameter
LOAD_PARITY        = 3'b101; parameter
FIFO_FULL_STATE    = 3'b110;
parameter LOAD_AFTER_FULL    = 3'b111;
reg [2:0] PS, NS;  always @(posedge clk)
begin
if (!resetn)
PS <= DECODE_ADDRESS;
```

```
else if (soft_rst_0 || soft_rst_1 || soft_rst_2)
PS <= DECODE_ADDRESS;
else PS
<= NS;
end
always @(*) begin NS =
DECODE_ADDRESS; case
(PS)
DECODE_ADDRESS: begin
if ((pkt_valid && din == 2'b00 && fifo_empty_0) ||(pkt_valid && din == 2'b01 &&
fifo_empty_1) ||(pkt_valid && din == 2'b10 && fifo_empty_2)) NS
= LOAD_FIRST_DATA;
else if ((pkt_valid && din == 2'b00 && !fifo_empty_0) ||(pkt_valid && din == 2'b01 &&
!fifo_empty_1) ||(pkt_valid && din == 2'b10 && !fifo_empty_2))
NS = WAIT_TILL_EMPTY;
else
NS = DECODE_ADDRESS; end
LOAD_FIRST_DATA: NS
= LOAD_DATA;
LOAD_DATA: begin
if (fifo_full)
NS = FIFO_FULL_STATE;
else if (!fifo_full && !pkt_valid) NS
= LOAD_PARITY;
else
NS = LOAD_DATA;
end
WAIT_TILL_EMPTY: begin
if ((din == 2'b00 && fifo_empty_0) ||(din == 2'b01 && fifo_empty_1) ||(din == 2'b10 &&
fifo_empty_2))
NS = LOAD_FIRST_DATA;
else
NS = WAIT_TILL_EMPTY; end
FIFO_FULL_STATE: begin
if (!fifo_full)
NS = LOAD_AFTER_FULL;
else
NS = FIFO_FULL_STATE; end
LOAD_AFTER_FULL: begin
if (!parity_done && !low_pkt_valid)
NS = LOAD_DATA;
else if (!parity_done && low_pkt_valid)
```

```
NS = LOAD_PARITY; else
if (parity_done)
NS = DECODE_ADDRESS;
end LOAD_PARITY:
NS = CHECK_PARITY_ERROR; CHECK_PARITY_ERROR:
begin
if (fifo_full)
NS = FIFO_FULL_STATE;
else
NS = DECODE_ADDRESS;
end endcase
end
assign detect_addr  = (PS == DECODE_ADDRESS);
assign wr_en_reg    = (PS == LOAD_DATA || PS == LOAD_PARITY || PS ==
LOAD_AFTER_FULL);
assign ld_state     = (PS == LOAD_DATA); assign laf_state    = (PS ==
LOAD_AFTER_FULL); assign lfd_state    = (PS == LOAD_FIRST_DATA); assign
full_state   = (PS == FIFO_FULL_STATE); assign rst_int_reg  = (PS ==
CHECK_PARITY_ERROR); assign busy        = (PS == LOAD_FIRST_DATA || PS ==
LOAD_PARITY ||PS == FIFO_FULL_STATE || PS == LOAD_AFTER_FULL ||PS ==
WAIT_TILL_EMPTY || PS ==
CHECK_PARITY_ERROR); endmodule
```

TEST BENCH:

```
module router_fsm_tb;

reg clk = 0; reg resetn;
reg pkt_valid, fifo_full;
reg fifo_empty_0, fifo_empty_1, fifo_empty_2;
reg soft_rst_0, soft_rst_1, soft_rst_2; reg
parity_done, low_pkt_valid;
reg [1:0] din;

wire wr_en_reg, detect_addr, ld_state, laf_state, lfd_state; wire
full_state, rst_int_reg, busy;

router_fsm dut (
  .clk(clk), .resetn(resetn),
  .pkt_valid(pkt_valid), .fifo_full(fifo_full),
  .fifo_empty_0(fifo_empty_0), .fifo_empty_1(fifo_empty_1), .fifo_empty_2(fifo_empty_2),
.soft_rst_0(soft_rst_0), .soft_rst_1(soft_rst_1), .soft_rst_2(soft_rst_2),
  .parity_done(parity_done), .low_pkt_valid(low_pkt_valid),
  .din(din),
  .wr_en_reg(wr_en_reg), .detect_addr(detect_addr), .ld_state(ld_state),
  .laf_state(laf_state), .lfd_state(lfd_state),
  .full_state(full_state), .rst_int_reg(rst_int_reg), .busy(busy)
);
```

```
always #5 clk = ~clk;

task apply_reset; begin
resetn = 0;   soft_rst_0 = 0;
soft_rst_1 = 0;   soft_rst_2 =
0;   repeat (2) @(posedge
clk);
  resetn      =      1;
@(posedge   clk);
end
endtask

task idle_defaults;
begin   pkt_valid =
0;   fifo_full = 0;
fifo_empty_0 = 0;
fifo_empty_1 = 1;
fifo_empty_2 = 1;
parity_done = 0;
low_pkt_valid = 0;
din = 2'b00; end
endtask

initial begin
  $dumpfile("router_fsm_tb.vcd");
  $dumpvars(0, router_fsm_tb);

  $display("------------------------------------------------------------");
  $display("TIME  PS detect_addr lfd ld laf full low_pkt rst_int busy");
  $monitor("%0t   %b     %b      %b   %b   %b   %b   %b     %b      %b",
$time, dut.PS, detect_addr, lfd_state, ld_state, laf_state,         full_state,
low_pkt_valid, rst_int_reg, busy);
  $display("------------------------------------------------------------");   idle_defaults();
apply_reset();

  pkt_valid     =     1;
din      =      2'b10;
fifo_empty_2 = 1;
  @(posedge clk);

  fifo_full = 1;
  @(posedge clk);
  fifo_full = 0;
```

```
 @(posedge clk);

 low_pkt_valid = 0;
parity_done   =   0;
@(posedge     clk);
pkt_valid     =     0;
@(posedge clk);
 @(posedge clk);
 @(posedge clk);

 #20 $finish;
end endmodule
```

A.3 Register

```
module register (input clk,input reset,input pkt_valid,input [7:0] d_in,input fifo_full,input
detect_add,input ld_state,input laf_state,input full_state,input lfd_state,input
reset_int_reg,output reg [7:0] d_out,output reg err,output reg parity_done,output reg
low_pkt_valid);

reg [7:0] header_byte, internal_reg, internal_parity, packet_parity; always
@(posedge clk) begin
if (!reset) parity_done <= 0; else if (detect_add) parity_done <= 0; else if ((ld_state
&& (~fifo_full) && (~pkt_valid)) || (laf_state && low_pkt_valid &&
(~parity_done)))
parity_done <= 1; end always
@(posedge clk) begin if
(!reset) low_pkt_valid <= 0;
else if (reset_int_reg)
low_pkt_valid <= 0; else if
(ld_state && ~pkt_valid)
low_pkt_valid <= 1; end
always @(posedge clk) begin if (!reset) begin d_out
<= 8'd0; header_byte   <= 8'd0; internal_reg  <= 8'd0;
end else if (detect_add && pkt_valid && d_in[1:0] !=
2'b11) header_byte <= d_in; else if (lfd_state) d_out <=
header_byte; else if (ld_state && ~fifo_full) d_out <=
d_in; else if (ld_state && fifo_full) internal_reg <= d_in;
else if (laf_state) d_out <= internal_reg; end always
@(posedge clk) begin
if (!reset) internal_parity <= 8'd0; else if
(detect_add) internal_parity <= 8'd0; else if
(lfd_state && pkt_valid) internal_parity <=
internal_parity ^ header_byte; else if (ld_state
&& pkt_valid && ~full_state) internal_parity <=
internal_parity ^ d_in; else internal_parity <=
```

```
internal_parity; end always @(posedge clk)
begin
if (!reset) err <= 0; else if
(parity_done) begin if
(internal_parity ==
packet_parity) err <= 0;
else err <= 1; end else err
<= 0; end always
@(posedge clk) begin
if (!reset) packet_parity <= 8'd0; else if (detect_add) packet_parity <= 8'd0;
else if ((ld_state && !fifo_full && !pkt_valid) || (laf_state && ~parity_done &&
low_pkt_valid)) packet_parity <= d_in; end endmodule
```

TEST BENCH:

```
module register_tb;

reg clk, resetn, pkt_valid, fifo_full, detect_add, ld_state, laf_state, full_state, lfd_state,
reset_int_reg; reg [7:0] d_in; wire err, parity_done, low_pkt_valid; wire [7:0] d_out;

  register DUT
(.clk(clk),.reset(~resetn),.pkt_valid(pkt_valid),.d_in(d_in),.fifo_full(fifo_full),.detect_add(de
tect_add),.ld_state(ld_state),.laf_state(laf_state),.full_state(full_state),.lfd_state(lfd_state),.r
eset_int_reg(reset_int_reg),.d_out(d_out),.err(err),.parity_done(parity_done),.low_pkt_vali
d(low_pkt_valid));

always #5 clk = ~clk; task
send_byte(input [7:0] data);
begin d_in = data;
#10; end endtask
initial begin clk =
0; resetn = 0;
pkt_valid = 0;
fifo_full = 0;
detect_add = 0;
ld_state = 0;
laf_state = 0;
full_state = 0;
lfd_state = 0;
reset_int_reg = 0;
d_in = 0; #12
resetn = 1;
```

```
pkt_valid = 1;
detect_add = 1;
send_byte(8'hA5);
detect_add = 0;
lfd_state = 1; #10
lfd_state = 0;
ld_state = 1;
fifo_full = 0;
send_byte(8'h11); send_byte(8'h22);
fifo_full = 1;
send_byte(8'h33);
laf_state = 1; #10
laf_state = 0;
fifo_full = 0;
pkt_valid = 0;
ld_state = 1;
send_byte(8'h55);
ld_state = 0;
#40;if (err)
$display("? TEST PASSED: Parity mismatch detected as expected. ERR=%b", err); else
$display("? TEST FAILED: ERR should be 1 here but is 0."); #50
$finish;
end initial
begin
$monitor("T=%0t | d_in=%h | d_out=%h | parity_done=%b | err=%b",$time, d_in, d_out,
parity_done, err); end
endmodule
```

A.4 Synchronizer

```
module router_sync (input clk,input resetn,input [1:0] data_in,input detect_addr,input
full_0,input full_1,input full_2,input empty_0,input empty_1,input empty_2,input
wr_en_reg,input read_enb_0, input read_enb_1,input read_enb_2,output reg [2:0]
write_enb,output reg fifo_full,output vld_out_0,output vld_out_1,output vld_out_2,output
reg soft_reset_0,output reg soft_reset_1,output reg soft_reset_2); reg [1:0] tmp_din; reg
[5:0] count0, count1, count2; always @(posedge clk or negedge resetn) begin if (!resetn)
tmp_din <= 2'b00; else if (detect_addr) tmp_din <= data_in;
end
always @(*) begin case
(tmp_din) 2'b00: begin
fifo_full = full_0; write_enb = (wr_en_reg) ? 3'b001 :
3'b000; end 2'b01: begin
fifo_full = full_1;
```

```
write_enb = (wr_en_reg) ? 3'b010 : 3'b000; end
2'b10: begin
fifo_full = full_2; write_enb = (wr_en_reg) ? 3'b100 :
3'b000; end

default: begin fifo_full =
1'b0; write_enb =
3'b000; end endcase
end
assign vld_out_0 = ~empty_0; assign vld_out_1 =
~empty_1; assign vld_out_2 = ~empty_2; always
@(posedge clk or negedge resetn) begin if (!resetn) begin
count0 <= 0; soft_reset_0 <= 0; end else if (vld_out_0)
begin if (!read_enb_0) begin if (count0 == 29) begin
soft_reset_0 <= 1; count0 <= 0; end else begin soft_reset_0
<= 0; count0 <= count0 + 1; end end else begin count0 <=
0; soft_reset_0 <= 0; end end else begin count0 <= 0;
soft_reset_0 <= 0; end end always @(posedge clk or
negedge resetn) begin
if (!resetn) begin
count1 <= 0;
soft_reset_1 <= 0;
end else if
(vld_out_1) begin if
(!read_enb_1) begin
if (count1 == 29)
begin soft_reset_1 <=
1; count1 <= 0; end
else begin
soft_reset_1 <= 0;
count1 <= count1 +
1; end end else begin
count1 <= 0;
soft_reset_1 <= 0;
end end else begin
count1 <= 0;
soft_reset_1 <= 0;
end end
always @(posedge clk or negedge resetn) begin if
(!resetn) begin count2 <= 0; soft_reset_2 <= 0; end else if
(vld_out_2) begin if (!read_enb_2) begin if (count2 == 29)
begin soft_reset_2 <= 1; count2 <= 0; end else begin
soft_reset_2 <= 0; count2 <= count2 + 1; end end else
begin count2 <= 0;
```

```
soft_reset_2 <= 0;
end end else begin
count2 <= 0;
soft_reset_2 <= 0;
end end
endmodule
```

TEST BENCH:

```
module router_sync_tb; reg [1:0] data_in; reg clk, resetn,
detect_addr, full_0, full_1, full_2; reg empty_0, empty_1,
empty_2, wr_en_reg; reg read_enb_0, read_enb_1,
read_enb_2; wire [2:0] write_enb; wire fifo_full, vld_out_0,
vld_out_1, vld_out_2; wire soft_reset_0, soft_reset_1,
soft_reset_2; router_sync DUT (.clk(clk), .resetn(resetn),
.detect_addr(detect_addr),.wr_en_reg(wr_en_reg),
.data_in(data_in),.full_0(full_0), .full_1(full_1),
.full_2(full_2),.empty_0(empty_0), .empty_1(empty_1),
.empty_2(empty_2),.read_enb_0(read_enb_0), .read_enb_1(read_enb_1),
.read_enb_2(read_enb_2),.write_enb(write_enb),
.fifo_full(fifo_full),.vld_out_0(vld_out_0), .vld_out_1(vld_out_1),
.vld_out_2(vld_out_2),.soft_reset_0(soft_reset_0),
.soft_reset_1(soft_reset_1), .soft_reset_2(soft_reset_2)); always #5
clk = ~clk; task apply_reset; begin resetn = 0; #(20); resetn = 1;
end endtask task write_to_fifo(input [1:0] addr); begin data_in =
addr; detect_addr = 1;
wr_en_reg = 1;
#(10); detect_addr
= 0; wr_en_reg = 0;
end endtask
task fifo_status(input f0, f1, f2, e0, e1, e2); begin
full_0 = f0; full_1 = f1; full_2 = f2; empty_0 = e0;
empty_1 = e1; empty_2 = e2;
#(10); end endtask task
generate_soft_reset(input sel); integer i;
begin case (sel) 0: begin read_enb_0 = 0;
empty_0 = 0;
for (i = 0; i < 35; i = i + 1) #(10); end
1: begin read_enb_1 = 0; empty_1 = 0;
for (i = 0; i < 35; i = i + 1) #(10); end
2: begin read_enb_2 = 0; empty_2 = 0;
for (i = 0; i < 35; i = i + 1) #(10); end
endcase end endtask initial begin clk
= 0; detect_addr = 0; wr_en_reg = 0;
data_in = 0; read_enb_0 = 0;
read_enb_1 = 0; read_enb_2 = 0;
```

```
fifo_status(0, 0, 0, 1, 1, 1);
apply_reset; fifo_status(0, 0, 0,
0, 1, 1); write_to_fifo(2'b00);
fifo_status(0, 0, 0, 1, 0, 1);
write_to_fifo(2'b01);
fifo_status(0, 0, 0, 1, 1, 0);
write_to_fifo(2'b10);
fifo_status(1, 0, 0, 1, 1, 1);
write_to_fifo(2'b00);
generate_soft_reset(0);
read_enb_0 = 1; #(50);
generate_soft_reset(1);
#(100);
$finish; end
initial begin
$monitor("Time=%0t din=%b write_enb=%b fifo_full=%b
vld_out={%b,%b,%b} soft_reset={%b,%b,%b}",$time, data_in, write_enb,
fifo_full,vld_out_0, vld_out_1, vld_out_2,soft_reset_0, soft_reset_1, soft_reset_2); end
endmodule
```

A.5 Top module

```
module router_1x3 (input clk,input resetn,input pkt_valid,input [7:0] data_in,input
read_enb_0,input read_enb_1,input read_enb_2,output vld_out_0,output vld_out_1,output
vld_out_2,output err,output busy,output [7:0] data_out_0,output [7:0] data_out_1,output [7:0]
data_out_2);


  wire soft_reset_0, full_0, empty_0;    wire
soft_reset_1, full_1, empty_1;    wire
soft_reset_2, full_2, empty_2;    wire fifo_full,
detect_addr, ld_state, laf_state;    wire
full_state, lfd_state, reset_int_reg;    wire
parity_done, low_pkt_valid, wr_en_reg;    wire
[2:0] write_enb;    wire [7:0] reg_dout;

  fifo_16x9 FIFO_0 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_0),
    .write_enb(write_enb[0]),
```

```
    .read_enb(read_enb_0),
    .lfd(lfd_state),
    .data_in(reg_dout),
    .data_out(data_out_0),
    .full(full_0),
    .empty(empty_0)
);

fifo_16x9 FIFO_1 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_1),
    .write_enb(write_enb[1]),
    .read_enb(read_enb_1),
    .lfd(lfd_state),
    .data_in(reg_dout),
    .data_out(data_out_1),
    .full(full_1),
    .empty(empty_1)
);

fifo_16x9 FIFO_2 (
    .clk(clk),
    .resetn(resetn),
    .soft_reset(soft_reset_2),
    .write_enb(write_enb[2]),
    .read_enb(read_enb_2),
    .lfd(lfd_state),
    .data_in(reg_dout),
    .data_out(data_out_2),
    .full(full_2),
    .empty(empty_2)
);

router_sync SYNC (
    .clk(clk),
    .resetn(resetn),
    .detect_addr(detect_addr),
    .wr_en_reg(wr_en_reg),
    .data_in(data_in[1:0]),
    .full_0(full_0),
    .full_1(full_1),
    .full_2(full_2),
    .empty_0(empty_0),
    .empty_1(empty_1),
    .empty_2(empty_2),
    .read_enb_0(read_enb_0),
```

```
    .read_enb_1(read_enb_1),
    .read_enb_2(read_enb_2),
    .write_enb(write_enb),
    .fifo_full(fifo_full),
    .vld_out_0(vld_out_0),
    .vld_out_1(vld_out_1),
    .vld_out_2(vld_out_2),
    .soft_reset_0(soft_reset_0),
    .soft_reset_1(soft_reset_1),
    .soft_reset_2(soft_reset_2)
);

register REG (
    .clk(clk),
    .reset(resetn),
    .pkt_valid(pkt_valid),
    .d_in(data_in),
    .fifo_full(fifo_full),
    .detect_add(detect_addr),
    .ld_state(ld_state),
    .laf_state(laf_state),
    .full_state(full_state),
    .lfd_state(lfd_state),
    .reset_int_reg(reset_int_reg),
    .d_out(reg_dout),
    .err(err),
    .parity_done(parity_done),
    .low_pkt_valid(low_pkt_valid)
);

router_fsm FSM (
    .clk(clk),
    .resetn(resetn),
    .pkt_valid(pkt_valid),
    .fifo_full(fifo_full),
    .fifo_empty_0(empty_0),
    .fifo_empty_1(empty_1),
    .fifo_empty_2(empty_2),
    .soft_rst_0(soft_reset_0),
    .soft_rst_1(soft_reset_1),
    .soft_rst_2(soft_reset_2),
    .parity_done(parity_done),
    .low_pkt_valid(low_pkt_valid),
```

```
    .din(data_in[1:0]),
    .wr_en_reg(wr_en_reg),
    .detect_addr(detect_addr),
    .ld_state(ld_state),
    .laf_state(laf_state),
    .lfd_state(lfd_state),
    .full_state(full_state),
    .rst_int_reg(reset_int_reg),
    .busy(busy)
  );

endmodule
```

TEST BENCH:

```
module router_1x3_tb();

reg clk = 0;                    reg
resetn;
reg pkt_valid;
reg read_enb_0, read_enb_1, read_enb_2;
reg [7:0] data_in;
wire vld_out_0, vld_out_1, vld_out_2;
wire err, busy;
wire [7:0] data_out_0, data_out_1, data_out_2;
integer i;  router_1x3 dut
(.clk(clk),.resetn(resetn),.pkt_valid(pkt_valid),.data_in(data_in),.read_enb_0(read_enb_0),.rea
d_enb_1(read_enb_1),.read_enb_2(read_enb_2),.data_out_0(data_out_0),.data_out_1(data_out
_1),.data_out_2(data_out_2),.vld_out_0(vld_out_0),.vld_out_1(vld_out_1),.vld_out_2(vld_out_2)
,.err(err),.busy(busy)); always
#5 clk = ~clk;  task
payload_XB;
input [6:0] len;          input
read;              input simRead;
input [1:0] addr;          input
[7:0] ext_parity;     reg [7:0]
header, data, parity;
begin parity = 0;
wait(!busy) begin
@(negedge clk);
pkt_valid = 1; header =
{len, addr}; data_in =
header; parity = parity ^
data_in; end @(negedge
clk);
for (i = 0; i < len; i = i + 1)
begin wait(!busy) begin
```

```
@(negedge clk); data =
{$random} % 256; data_in =
data; parity = parity ^ data_in;
end if (read && simRead && i == 5)
begin read_enb_0 = (addr == 2'b00);
read_enb_1 = (addr == 2'b01);
read_enb_2 = (addr == 2'b10); end
end wait(!busy) begin @(negedge
clk); pkt_valid = 0;
data_in = (ext_parity == 0) ? parity : ext_parity;
end if (read) begin @(negedge clk);
@(negedge clk); if (addr == 2'b00)
begin read_enb_0 = simRead ?
read_enb_0 : 1;
wait(dut.FIFO_0.empty); read_enb_0 =
0; end else if (addr == 2'b01) begin
read_enb_1 = simRead ? read_enb_1 :
1; wait(dut.FIFO_1.empty); read_enb_1
= 0; end else if (addr == 2'b10) begin
read_enb_2 = simRead ? read_enb_2 :
1; wait(dut.FIFO_2.empty); read_enb_2
= 0; end end end endtask initial begin
pkt_valid = 0; read_enb_0 = 0;
read_enb_1 = 0; read_enb_2 = 0;
data_in = 8'd0;
resetn = 0;
#10 resetn = 1;
#20 payload_XB(6'd8, 1, 0, 2'b00, 0);
#30; resetn = 0; #10; resetn = 1;
#20 payload_XB(6'd20, 1, 1, 2'b01, 0);
#30; resetn = 0; #10; resetn = 1;
#20 payload_XB(6'd16, 0, 0, 2'b01, 0);
#30; resetn = 0; #10; resetn = 1;
#10; read_enb_2
= 1;    #20;
read_enb_2 = 0;
#20 payload_XB(6'd14, 1, 0, 2'b10, 8'h25);
#100;
$display("\nSimulation finished at time %0t", $time);
$finish;
end
endmodule
```